

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO4 – Articulation (BL3)	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	30% of CIA	Total Marks:	100
CO4 Statement:	Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data.		
Student Name:	Sree B	Reg. No.:	192524023
Section / Batch:	AI and DS / 2025	Date:	26/08/2026

Code of Conduct

I Sree B (192524023) (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: B. S

Instructions

- This test contains 5 Comparative Analysis questions. Total: 100 Marks.
- When a student answer using comparative analysis should be based on four requirements: time complexity, space complexity advantages & limitations, real-world scenarios and operations.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Hashing	Linear Probing & Separate Chaining	1	20
Sorting	Merge Sort & Quick Sort	2	20
Hashing	Quadratic Probing and Double Hashing	3	20
Sorting Algorithms	Complexity Analysis	4	20
Quick Sort	Recursive and Iterative implementations	5	20
GRAND TOTAL:			100 Marks

Annexure A – Comparative Analysis

1. Compare Linear Probing and Separate Chaining for collision handling in hashing based on: (20 Marks)
 - a) Clustering effect
 - b) Memory usage
 - c) Performance under high load factor
 - d) Which method is more suitable for large-scale applications and why?
2. Compare quick sort and merge sort for sorting a large linked list of 10 million nodes. Analyze memory requirements (in-place vs. extra space), cache behavior, and time complexity. Which algorithm is preferable for linked lists and why? How does the answer change for arrays? (20 Marks)
3. Compare Quadratic Probing and Double Hashing in terms of: (20 Marks)
 - a) Clustering issues
 - b) Collision resolution efficiency
 - c) Suitability for dense hash tables
4. Compare Best-case and Worst-case time complexity of sorting algorithms based on the following: (20 Marks)
 - a) Practical significance
 - b) Impact on algorithm selection
5. Compare Recursive and Iterative implementations of Quick Sort in terms of: (20 Marks)
 - a) Space complexity (stack usage)
 - b) Ease of implementation
 - c) Risk of stack overflow

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Understanding	20	Demonstrates complete and accurate understanding of all data structures with advanced insights	Shows clear understanding with minor gaps	Basic understanding; some misconceptions present	Limited or incorrect understanding
Comparative Analysis Depth	20	Thorough, multi-dimensional comparison with strong justification and critical evaluation	Good comparison with relevant factors considered	Limited comparison; lacks depth or justification	Minimal or incorrect comparison
Use of Parameters (Time/Space/Operations)	30	All parameters analysed accurately with complexity discussion	Most parameters covered correctly	Few parameters considered; some inaccuracies	Parameters missing or incorrect
Critical Thinking	20	Strong justification of why one structure is better in given contexts	Some justification present	Limited reasoning	No critical reasoning
Clarity	10	Exceptionally well-structured, logical flow, clear tables/diagrams	Well-organized with minor issues	Some organization issues; lacks clarity	Poor structure and readability

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1) Linear Probing vs Separate Chaining

Factors	Linear Probing	Separate Chaining
1. Collision Handling	Search next available slot	Stores collided keys in a linked list/bucket
2. Clustering	High Primary clustering	No Primary clustering
3. Memory usage	Lower; uses table itself	Higher, requires extra nodes / pointers.
4. Cache performance	Usually very good	Generally poor due to pointer traversal
5. Load Factor	Performance decreases rapidly when high	Can work reasonably even when load factor > 1
6. Average Search	$O(1)$ when table is lightly loaded	$O(1)$ with good hash function
7. Worst case Search	$O(n)$	$O(n)$
8. Insertion	$O(1)$ average	$O(1)$ average
9. Deletion	More complicated	Relatively easy
10. Implementation	Simple	Requires linked-list or bucket management

a) Clustering effect

(i) Linear probing suffers from primary clustering. i.e., several consecutive positions become occupied, new collisions cause longer consecutive groups

Example: Hash table [10] [20] [30] [] [] []

If another key maps to position 1

[10] [20] [30] [40] [] []

The cluster continues to grow.

(ii) Separate chaining does not suffer from primary clustering because each table position maintains a separate chain

Example: Index 0 $\rightarrow$ 15 $\rightarrow$ 25

Index 1 $\rightarrow$ 11

Index 2 $\rightarrow$ 32 $\rightarrow$ 42 $\rightarrow$ 52

(b) Memory usage

Linear probing is memory efficient because all elements are stored directly inside the hash table

Separate chaining requires additional memory for linked-list nodes or buckets structures

Therefore: Linear $\rightarrow$ less extra memory

Separate chaining $\rightarrow$ more memory but flexible

(C) Performance under high load factor

$$\text{Load factor} : \alpha = \frac{n}{m}$$

where n = number of stored elements

m = table size.

- * Linear probing becomes inefficient as α approaches 1 because finding an empty position requires many probes.
- * Separate chaining handles high load factors better because multiple elements can occupy same bucket.
- 1) which is suitable for large scale applications!

Separate chaining is generally more suitable for large scale applications, especially when the number of elements is unpredictable.

Reasons:

- * Handles high load factors better.

- * Collision management is flexible

- * Table does not need to be nearly empty.

- * Insertion and deletion are relatively straightforward.

- * Hash tables can grow dynamically

However linear probing can be preferable when memory locality and cache performance are extremely important such as compact in-memory hash tables.

② Quick Sort vs Merge Sort for a Linked List of 10 Million Nodes

Factors	Quick Sort	Merge Sort
Average Time	$O(n \log n)$	$O(n \log n)$
Worst-case Time	$O(n^2)$	$O(n \log n)$
Best case	$O(n \log n)$	$O(n \log n)$
Extra space for linked list	$O(\log n)$ average recursion	$O(\log n)$ recursion, merges can be pointer-based
Stability	Usually not stable	Stable
Linked List suitability	Moderate	Excellent
Sequential access	Less natural	Excellent
Cache behaviour	Depends on partitioning	Sequential traversal is predictable
Worst-case guarantee	No	Yes

Quick Sort on Linked List

Quick Sort works by:

1. selecting a pivot
2. Partitioning elements around the pivot.
3. Recursively sorting the partitions.

For a well balanced partitioning! $T(n) = 2T(n/2) + O(n)$

Therefore

$$\therefore T_n = O(n \log n)$$

But if the pivot repeatedly produces highly unbalanced partitions! $T(n) = T(n-1) + O(n)$

Therefore! $T(n) = O(n)^2$

For 10 million nodes, this worst case can be extremely expensive

Merge Sort on Linked List

Merge Sort divides the Linked List into two halves, recursively sorts each half, and merges the sorted lists.

~~10 million nodes~~ $\rightarrow$ 5 million $\rightarrow$ 2.5 million $\dots \rightarrow$ Merge sorted list
5 million $\quad$ 2.5 million

Its complexity is $O(n \log n)$ is the best, average and worst cases.

Why Merge Sort is better for Linked Lists

- * Linked List do not support direct random access efficiently
- * Merge Sort only requires sequential traversal and can merge lists by changing pointers
- * Therefore, Merge Sort is generally preferable for a linked list of 10 million nodes.

Cache Behaviour

- * Linked List nodes may be scattered in memory, so cache behaviour is already less favourable than arrays.
 - * Merge Sort has a predictable sequential access pattern.
 - * Quick Sort's partitioning can involve repeated pointer traversal and is more dependent on pivot selection
- What about arrays?

For arrays, the answer changes.

Quick Sort is often preferred for arrays because:

- * It can be performed in-place
- * It has good cache locality.
- * It generally requires little extra memory
- * Average performance is excellent

But Merge Sort is preferred when

- * Stability is required
- * A guaranteed $O(n \log n)$ performance is needed
- * Extra memory is acceptable

Final Choice

Linked List $\rightarrow$ Merge Sort

Arrays $\rightarrow$ Usually Quick Sort, when average performance and memory efficiency are pointers

Quadratic Probing vs Double Hashing

Quadratic Probing

When a collision occurs quadratic probing searches position using

$$h(k, i) = (h(k) + i^2) \bmod m$$

where $i = 0, 1, 2, 3, \dots$

Example: First collision $\rightarrow h(k) + (1)^2$

Second collision $\rightarrow h(k) + (2)^2$

Third collision $\rightarrow h(k) + (3)^2$

Double Hashing

Double Hashing uses two hash functions:

$$h(k, i) = (h_1(k) + i h_2(k)) \bmod m$$

The second hash function determines the step size

Factors	Quadratic Probing	Double Hashing
1. Collision method	Quadratic Jumps	Uses second hash function
2. Primary clustering	Avoids it	Avoids it
3. Secondary clustering	Can occur	Much less likely
4. Collision efficiency	Good	Very Good
5. Hash functions	One main hash function	Two hash functions
6. Implementation	Simpler	More complex
7. Dense Table	Moderate suitability	Better suitability
8. Performance	Degrades at high load	Generally better under collisions

a) Clustering Issues

Quadratic Probing: Quadratic probing eliminates ^{primary} clustering, but it can suffer from secondary clustering. Keys having the same initial hash value follow the same probing sequence.

Double Hashing: Double Hashing greatly reduces clustering because different keys can have different probe sequences. Therefore Double Hashing $\rightarrow$ better clustering behaviour.

b) Collision Resolution efficiency

* Quadratic Probing provides a systematic way of finding empty positions.

* Double hashing generally provides better distribution of probes.

Therefore Double hashing is usually more efficient when many collisions occur.

c) Suitability for dense hash tables

As the load factor becomes high, collisions become more frequent. Quadratic probing may experience secondary clustering. Double hashing distributes probes more evenly.

Therefore Double hashing is generally better suited for dense hash tables.

Best Case vs Worst - Case Time Complexity of Sorting Algorithms

Algorithm	Best Case	Average Case	Worst Case
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

a) Practical Significance

Best Case Complexity

Best - case complexity describes the minimum amount of work under favourable input condition.

For example, optimized Bubble sort and Insertion Sort can achieve, when the input is already sorted or nearly sorted.

This is useful when the application frequently receives nearly sorted data.

Worst - case complexity

Worst - case complexity describes the maximum amount of work that may be required.

For example: Quick sort can become, when poor pivot repeatedly create highly unbalanced partitions.

Worst-case analysis is particularly important for systems where predictable performance is required.

Example: For $n = 1,000,000$:

$$n \log_2 n \approx 20,000,000,$$

$$\text{while } n^2 = 1,000,000,000,000.$$

This demonstrates why an algorithm with an $O(n^2)$ worst case can become impractical for very large datasets.

b) Impact on algorithm selection

Algorithm selection should depend on:

- * Input size
- * Data characteristics
- * Required execution time
- * Available memory
- * Stability requirements
- * Worst-case guarantees
- * Whether data is nearly sorted
- * Whether predictable performance is required

examples: Small or nearly sorted data:

→ Insertion Sort

Large data requiring guaranteed $O(n \log n)$,

→ Merge Sort or Heap Sort

Large arrays where average speed is important:

→ Quick Sort

External sorting:

→ Merge Sort

5. Recursive vs Iterative Quick Sort

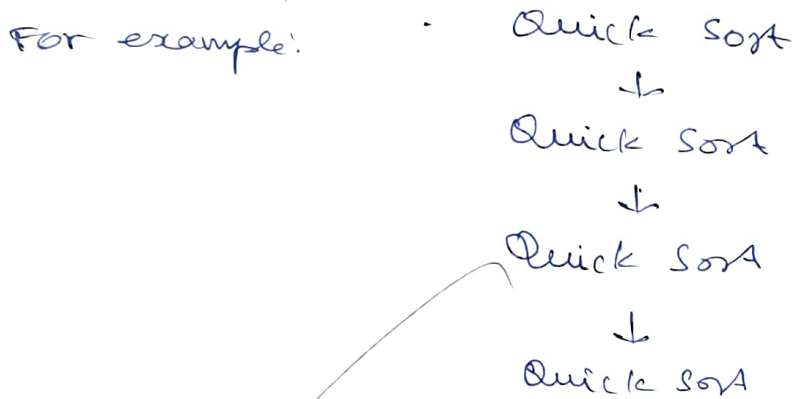
Factor	Recursive Quick Sort	Iterative Quick Sort
Implementation	Easier and more natural	More complex
Stack usage	Function call stack	Explicit stack
Average space	$O(\log n)$	$O(\log n)$ with controlled stack
Worst-case space	$O(n)$	Can also reach $O(n)$ if poorly managed
Stack overflow risk	Higher	Lower if stack is controlled
Code readability	Simple	Slightly more complex
Partitioning	Recursive calls	Explicit stack operations

a) Space complexity and stack usage

Recursive version: A recursive Quick Sort calls itself for the subarray

Balanced partition: $O(\log n) \rightarrow$ recursion depth

Worst-case partition: $O(n) \rightarrow$ recursion depth



This can consume a large amount of stack memory

Iterative version:

The iterative version uses an explicit stack data structure

Instead of recursive function calls it stores subarray boundaries (left, right) in a stack. This gives the programmer more control over memory usage.

Case of Implementation

Recursive Quick Sort is simpler:

Quick Sort (array, low, high)

partition array

Quick Sort (left)

Quick Sort (right)

The recursive structure naturally matches the divide and conquer design.

Iterative Quick Sort requires:

1. Creating a stack.
2. Pushing subarray boundaries.
3. Popping range.
- ↳ Partitioning them
- ↳ Pushing remaining ranges

Therefore Recursive $\rightarrow$ easier

Iterative $\rightarrow$ more control but more complicated

(c) Risk of stack overflow

Recursive Quick Sort can suffer stack overflow when partitions become highly unbalanced.

Worst - case recursion depth: $O(n)$

Iterative Quick Sort avoids using the program's function call stack for recursion.

Therefore it can significantly reduce stack-overflow risk.

Important Optimization

Even in an iterative implementation, pushing both subarrays blindly can consume unnecessary memory. A common optimization is to process the smaller partition first and keep the larger partition for later, reducing maximum stack requirements.

Final Comparison

For ordinary applications:

Recursive Quick Sort is preferred for simplicity and readability.

For very large datasets or systems where stack usage must be controlled.

✓ Iterative Quick Sort is preferable